

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	7
1.1 Анализ предметной области.....	7
1.2 Описание вариантов решения проблемы	7
1.3 Описание требований к разрабатываемому решению	8
2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ	11
2.1 Выбор инструментов и / или средств для реализации проекта	13
2.2 Разработка проекта решения.....	13
2.2.1 Диаграмма UserStoryMap	19
2.2.2 Диаграмма UseCase.....	19
2.2.3 ER-диаграмма.....	21
2.2.4 Описание архитектуры фронтенда	22
2.2.5 Описание архитектуры бекенда	24
2.2.6 Список API	25
3 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	31
3.1 Разработка составных элементов.....	31
3.1.1 Пользовательский интерфейс.....	31
3.1.2 Реализация фронтенд части.....	36
3.1.3 Реализация бекенд части	36
3.1.4 Реализация базы данных.....	37
3.1.5 Интеграция модулей	40
3.2 Установка и настройка	40
3.3 Тестирование	40
3.4 Ввод в эксплуатацию	42
3.5 Сопроводительная документация	43
3.5.1 Назначение и область применения	45
3.5.2 Руководство пользователя.....	45

3.5.3 Руководство администратора	45
3.5.4 Регламент сопровождения	45
3.6 Вывод	46
ЗАКЛЮЧЕНИЕ	47
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48
ПРИЛОЖЕНИЕ	48

ВВЕДЕНИЕ

Актуальность работы: Современный рынок разработки программного обеспечения обязывает разработчиков программного обеспечения не только быть специалистами своей области, но и ориентироваться в большинстве смежных технологических областей. Данная работа посвящена разработке современного программного обеспечения в соответствии с поставленной задачей и темой работы. Такая деятельность, помимо формирования практического опыта, позволяет сформировать новое видение классических задач по разработке программных комплексов и программного обеспечения.

Цель работы: Разработка программного продукта система управления турфирмой «ТУРИСТ».

Задачи работы:

1. Провести анализ предметной области.
2. Проанализировать аналоги разрабатываемого решения.
3. Описать требования к разрабатываемому решению.
4. Выбрать инструменты и/или средства для реализации проекта.
5. Разработать проект решения.
6. Разработать программный продукт.
7. Составить отчет по работе.
8. Представить отчет к защите и ответить на вопросы по проекту.

Объектом исследования является процесс оперативного управления продажами туристических путёвок в туристической фирме, включающий учёт клиентов, путевок, продаж, а также формирование аналитических отчётов (подбор по городу, датам, «горящие» путевки, оценка скидок, спрос).

Предметом исследования является автоматизация учёта и анализа продажи стандартных туристических путёвок с реализацией функций добавления/удаления клиентов по путевке, фильтрации по городу и датам, расчёта потерь от скидок и определения наиболее востребованных путёвок в программной системе на языке java script.

В качестве основных методов исследования применены анализ существующих аналогов, синтез, сравнение и моделирование. Практическая реализация поставленной задачи соответствует основным подходам к разработке программного обеспечения.

Информационной базой исследования послужили открытые источники, в том числе доступные в сети Интернет, а также материалы курса «Создание программного обеспечения», доступные через систему дистанционного обучения РТУ МИРЭА.

В данном отчете будет представлен процесс разработки программного продукта, технологическое проектирование и описание системы, а также непосредственно результаты разработки.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

Анализ предметной области выполнен на основе технического задания на разработку программной системы для туристической фирмы. Предметной областью является процесс продажи стандартных путёвок, включающий учёт клиентов, хранение сведений о путевках и регистрацию продаж.

Каждая путёвка содержит стоимость, даты начала и окончания поездки, населённый пункт и дополнительные услуги. Клиент имеет ФИО, телефон, паспорт и персональную скидку, причём каждый клиент покупает только одну путёвку.

Фирма ведёт журнал продаж с фиксацией даты, названия путевки, количества, сотрудника и остатка.

Ключевые процессы, требующие автоматизации: ведение справочников, поиск клиентов по городу, проверка путевок по дате начала, выявление «горящих» путевок (отправление не позднее чем через 5 дней от текущей даты), расчёт потерь от скидок, определение самой дорогой путевки, анализ спроса, а также добавление и удаление клиентов по путевке.

При ручном учёте возникают высокие временные затраты на поиск, ошибки в расчётах, отсутствие оперативной аналитики. Пример инцидента: менеджер тратит 30–40 минут на поиск «горящих» путевок, ошибается и теряет клиента.

Проблема, решаемая в проекте, — отсутствие централизованной автоматизированной системы для эффективного хранения, поиска и анализа данных о клиентах, путевках и продажах.

Разработка системы позволит сократить время обработки запросов, повысить точность расчётов и улучшить качество обслуживания.

1.2 Описание вариантов решения проблемы

Для решения проблемы автоматизации учёта продаж туристических путёвок были рассмотрены три инструмента, каждый из которых представляет собой отдельный класс решений: табличный редактор, настольная СУБД общего назначения и профессиональная туристическая CRM-система.

1. Microsoft Excel

Что делает (какие проблемы решает):

- позволяет вести справочники клиентов, путёвок и продаж в виде таблиц;
- реализует базовый поиск и фильтрацию (по городу, датам, ФИО);
- возможность рассчитывать стоимость с учётом скидки с помощью ручных формул;
- может найти «горящие» путёвки, если менеджер введёт условие сравнения дат;

Чего не решает (ограничения):

- нет автоматического выявления путёвок, отправляющихся не позднее чем через 5 дней от текущей даты — каждый раз нужно настраивать условие заново;
- расчёт потерь от скидок требует ручного создания отдельных формул и легко приводит к ошибкам;
- определение самой дорогой путёвки и анализ спроса по журналу продаж выполняются сложными формулами или макросами;
- при росте числа записей (более 1000) скорость работы падает, а фильтрация становится неудобной;
- отсутствует централизованное хранение — файлы могут дублироваться, теряться или повреждаться;

2. Microsoft Access

Что делает (какие проблемы решает):

- позволяет создать связанные таблицы (клиенты, путёвки, продажи) с поддержкой целостности данных;
- реализует автоматическую фильтрацию по датам и населённым пунктам через запросы;
- можно построить форму для добавления/удаления клиентов по путевке;
- возможность рассчитывать остатки путёвок в журнале продаж;

Чего не решает (ограничения):

- нет встроенных функций именно для туристической отрасли: выявление «горящих» путёвок каждый раз требует написания сложного запроса с вычислением разницы дат;
- расчёт потерь от скидок и построение отчёта «самая дорогая путёвка» — это нестандартные операции, программируемые вручную;
- система требует знания языка SQL или конструктора запросов; для рядового менеджера турфирмы это слишком сложно;
- при многопользовательской работе возникают конфликты блокировок, так как Access не рассчитан на серьёзную совместную работу;
- отсутствует удобная аналитика спроса (какие путевки чаще покупают) без дополнительной разработки форм и отчётов;

3. Профессиональная CRM для туризма (TravelCRM)

Какие проблемы решает:

- полноценный учёт клиентов, продаж, путевок с автоматизацией типовых операций;
- есть модуль «горящих» туров и автоматические уведомления;
- рассчитывает скидки и итоговую стоимость;
- ведёт историю взаимодействия с клиентами;

Что не решает:

- ориентирована на крупные агентства с большим штатом; для малой турфирмы функционально избыточна;
- высокая стоимость лицензии и плата за поддержку (часто от 50 000 руб. в год и выше) ;
- не позволяет гибко настроить специфические отчёты, требуемые в задании: например, расчёт потерь от скидок именно для постоянных клиентов с персональной скидкой или определение самой дорогой путёвки на текущий день в реальном времени;
- требует времени на обучение персонала и сложна в развёртывании;
- многие функции (анализ спроса по каждому населённому пункту, быстрый поиск клиента по городу) реализованы не так, как нужно конкретной фирме, и их нельзя дешево доработать;

Вывод: почему необходима разработка нового продукта

Как показал анализ, существующие аналоги не удовлетворяют одновременно трём ключевым требованиям проекта:

1. **Полнота охвата задач** — ни Excel, ни Access, ни готовая туристическая CRM не закрывают все необходимые функции (выявление «горящих» путёвок по точному правилу «5 дней» без ручных настроек, расчёт потерь от скидок, мгновенное определение самой дорогой путёвки, анализ спроса по городу и услугам).
2. **Простота для конечного пользователя** — менеджер турфирмы не должен писать SQL-запросы или сложные формулы Excel. Требуется интерфейс с кнопками: «Найти горящие», «Рассчитать потери от скидок», «Добавить клиента к путевке».
3. **Низкая стоимость внедрения и поддержки** — профессиональные CRM дороги и избыточны, а универсальные инструменты требуют постоянной ручной настройки, что приводит к ошибкам и временным потерям (как в инциденте, когда менеджер тратил 30–40 минут на поиск «горящих» путёвок).

Таким образом, разработка новой специализированной программной системы для туристической фирмы является обоснованной. Она позволит объединить минимально необходимый набор функций, автоматизировать расчёты и поиск, исключить ручные ошибки и при этом остаться доступной по цене внедрения, в отличие от громоздких CRM-систем.

1.3 Описание требований к разрабатываемому решению

На основе проведённого анализа предметной области и выявленных недостатков аналогов сформулированы основные требования к разрабатываемой программной системе.

Функциональные требования включают: редактирования и удаления записей о клиентах (ФИО, телефон, паспорт, скидка) и путевках (название, дата начала и окончания, город, услуги, цена); ведение журнала продаж с фиксацией даты продажи, названия путевки, количества, ФИО сотрудника и остатка путевок в наличии.

Реализацию поиска клиентов по заданному городу; проверку наличия путевок с заданной датой начала; автоматическое выявление «горящих» путевок, дата отправления которых не более чем на 5 дней превышает текущую дату; расчёт суммы потерь фирмы из-за скидок постоянным клиентам; определение самой дорогой путевки из имеющихся в продаже на текущий день; выявление путевок, пользующихся наибольшим спросом (по количеству продаж).

Нефункциональные требования: интуитивно понятный графический или консольный интерфейс, рассчитанный на сотрудника без технической подготовки; обеспечение целостности данных (например, невозможность удалить путевку, на которую есть ссылки в продажах).

Быстродействие — время отклика на запрос не более 2 секунд при объёме до 1000 записей; хранение данных в локальном файле или встроенной базе данных без необходимости установки дополнительного серверного ПО.

Детальное техническое задание с описанием структур данных, алгоритмов работы и интерфейса пользователя приведено в приложении к курсовой работе. Разрабатываемая система должна полностью соответствовать перечисленным требованиям, что позволит эффективно решить проблему автоматизации учёта продаж туристических путёвок в малой фирме.

2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

2.1 Выбор инструментов и / или средств для реализации проекта

Для реализации программной системы управления продажами туристических путёвок был проведён сравнительный анализ возможных технологий в трёх слоях: фронтенд (пользовательский интерфейс), бэкенд (серверная логика) и система управления базами данных.

1. Выбор технологии для фронтенда (пользовательский интерфейс)

Вариант 1: Чистый HTML/CSS/JavaScript без фреймворков

Какие проблемы решает:

1. Низкий порог входа, не требует изучения дополнительных инструментов.
2. Лёгкость и быстроедействие на простых страницах.
3. Полный контроль над кодом без лишних абстракций.

4. Какие проблемы не решает (почему не подходит):

5. При увеличении числа интерактивных элементов (таблицы с редактированием, вкладки, формы) код становится трудно поддерживать.
6. Отсутствие компонентного подхода приводит к дублированию логики (например, для отображения списка клиентов в нескольких местах).
7. Управление состоянием приложения (какие данные загружены, какая вкладка активна) реализуется вручную и подвержено ошибкам.
8. Сложно реализовать динамическое обновление таблиц без полной перезагрузки страницы.

Вариант 2: jQuery с Bootstrap

Какие проблемы решает:

1. Упрощает манипуляции с DOM и AJAX-запросы по сравнению с чистым JavaScript.
2. Bootstrap даёт готовые стили для таблиц, форм и вкладок.

3. Большое количество плагинов для датапикеров, поиска и модальных окон.

Какие проблемы не решает (почему не подходит):

1. Код быстро превращается в «спагетти» из колбэков и манипуляций с DOM.

2. Отсутствует модульность и повторное использование компонентов (например, нельзя просто взять и переиспользовать компонент «карточка путевки» в разных местах).

3. Управление состоянием при синхронных и асинхронных запросах становится хаотичным.

4. Современные требования к производительности и организации кода jQuery уже не соответствует.

Вариант 3: Angular (версии 2+)

Какие проблемы решает:

1. Полноценный фреймворк с компонентной архитектурой, Dependency Injection и двусторонним связыванием данных.
2. Встроенная маршрутизация и HTTP-клиент.
3. Строгая типизация через TypeScript снижает количество ошибок.

Какие проблемы не решает (почему не подходит):

1. Крутая кривая обучения — менеджеру турфирмы не важно, но разработчику потребуется много времени на освоение.
2. Избыточность для небольшого приложения: много «магии» и конфигураций.
3. Сборка проекта сложнее, чем в React.
4. Вес итогового бандла больше, а производительность для таблиц на 1000 записей может страдать.

Итоговый выбор: React с JSX — он даёт компонентный подход, лёгкое управление состоянием (useState, useEffect), удобную маршрутизацию через React Router, и при этом остаётся достаточно простым для реализации интерфейса с вкладками, таблицами и формами, идеально подходя для

сотрудника без технической подготовки за счёт понятного визуального разделения на компоненты.

2. Выбор технологии для бэкенда (серверная логика)

Вариант 1: Python + Flask (или Django)

Какие проблемы решает:

1. Простота написания REST API, богатый набор библиотек для работы с датами и расчётами.
2. Django имеет встроенную админку, которая могла бы ускорить разработку.
3. Читаемый код, хорошая документация.

Какие проблемы не решает (почему не подходит):

1. Для вычислений (потери от скидок, «горящие» путевки по дате) производительность Python может быть ниже, чем у Node.js.
2. Поддержка долгого соединения или одновременных запросов требует дополнительной настройки асинхронности (FastAPI, но это уже другой выбор).
3. В контексте курсовой работы Python добавляет ещё один язык в проект, что усложняет поддержку, если фронтенд пишет тот же разработчик.

Вариант 2: Java + Spring Boot

Какие проблемы решает:

1. Высокая надёжность, масштабируемость и производительность для больших нагрузок.
2. Строгая типизация и мощные инструменты для работы с БД (Spring Data JPA).
3. Промышленный стандарт для серьёзных проектов.

Какие проблемы не решает (почему не подходит):

1. Чрезмерная сложность и объём кода для небольшой туристической системы.
2. Долгое время разработки и компиляции.
3. Требуется много памяти и ресурсов на сервере.

4. Нецелесообразно для малой фирмы с объёмом данных до 1000 записей.

Вариант 3: PHP (чистый или Laravel)

Какие проблемы решает:

1. Простота развёртывания на любом хостинге.
2. Laravel предоставляет удобные инструменты для работы с БД (Eloquent ORM) и маршрутизацию.
3. Огромное количество готовых решений.

Какие проблемы не решает (почему не подходит):

1. Современные реалии веб-разработки требуют постоянного поддержания сессии, что для REST API не всегда удобно.
2. Асинхронная обработка запросов в чистом PHP реализована слабее, чем в Node.js.
3. Производительность при одновременных запросах на расчёт агрегаций (сумма потерь от скидок, максимум цены) может быть ниже.

Итоговый выбор: Node.js + Express.js — позволяет писать и фронтенд (React) и бэкенд на одном языке (JavaScript/TypeScript), что упрощает разработку. Express легко создаёт REST API с методами GET/POST/PUT/DELETE для работы с клиентами, путёвками и продажами. Асинхронная модель идеальна для операций с БД (запросы к PostgreSQL) и не блокирует поток при вычислениях по датам или агрегациям.

3. Выбор технологии для базы данных

Вариант 1: SQLite

Какие проблемы решает:

1. Не требует установки и настройки сервера — один файл.
2. Очень легковесный, идеален для небольшого объёма данных.
3. Поддерживает стандартный SQL и транзакции.

Какие проблемы не решает (почему не подходит):

1. Не поддерживает многопользовательский режим (при одновременной работе нескольких менеджеров возможны блокировки и потеря данных).
2. Плохо масштабируется при росте числа записей и сложных запросах (например, группировка по спросу с JOIN нескольких таблиц).
3. Отсутствие тонкой настройки прав доступа и ролей.
4. Не гарантирует целостность ссылок на уровне сервера при конкурентных изменениях.

Вариант 2: MySQL

Какие проблемы решает:

1. Широко распространён, много документации и инструментов.
2. Хорошая производительность на чтении, поддержка многопользовательского доступа.
3. Есть хранимые процедуры и триггеры.

Какие проблемы не решает (почему не подходит):

1. Для агрегаций типа «сумма потерь от скидок» по нескольким связанным таблицам MySQL иногда менее оптимален, чем PostgreSQL.
2. Строгость соблюдения стандартов SQL ниже (например, с обработкой NULL и внешних ключей в некоторых движках).
3. Работа с датами и интервалами (вычисление «горящих» путевок по разнице в днях) в PostgreSQL удобнее за счёт встроенных операторов и типов.

Вариант 3: MongoDB (NoSQL)

Какие проблемы решает:

1. Гибкая схема данных — можно хранить путевки со вложенными услугами и списками клиентов.
2. Быстрая разработка без миграций схем.

Какие проблемы не решает (почему не подходит):

1. Не поддерживает сложные JOIN-запросы, которые необходимы для анализа спроса (путёвки + продажи + клиенты).

2. Агрегации для расчёта потерь от скидок и определения самой дорогой путевки на текущий день реализуются через конвейер агрегации, что сложно и медленно.
3. Нет встроенной поддержки целостности «внешний ключ» на уровне БД (при удалении путевки, на которую есть ссылки в продажах, придётся всё контролировать в коде).
4. Не решает проблему транзакционной согласованности для журнала продаж и остатков.

Итоговый выбор: PostgreSQL — поддерживает все необходимые типы запросов (фильтрация по датам, расчёт разницы дней для «горящих» путевок, агрегации SUM, MAX, COUNT с группировкой для анализа спроса). Обеспечивает ссылочную целостность на уровне FOREIGN KEY, что гарантирует невозможность удалить путевку с активными продажами. Работает стабильно в многопользовательском режиме и при объёме до 1000 записей даёт отклик значительно быстрее 2 секунд.

Общий вывод по стеку технологий

Таким образом, комбинация **React (фронтенд) + Node.js/Express (бэкенд) + PostgreSQL (БД)** полностью удовлетворяет требованиям проекта:

1. Позволяет реализовать все функциональные требования (поиск клиентов по городу, проверку дат, «горящие» путёвки за 5 дней, расчёт потерь от скидок, самую дорогую путёвку, анализ спроса, добавление/удаление клиентов по путевке).
2. Обеспечивает целостность данных и быстродействие не более 2 секунд.
3. Даёт интуитивно понятный компонентный интерфейс, рассчитанный на сотрудника без технической подготовки.
4. Не требует установки дополнительного серверного ПО, кроме самого приложения (PostgreSQL может быть развёрнут локально или на том же хосте).

Данный выбор является оптимальным для малой туристической фирмы с точки зрения затрат на разработку, поддержки и производительности.

2.2 Разработка проекта решения

2.2.1 Диаграмма UserStoryMap

Для детального планирования разработки и приоритизации требований была построена карта пользовательских историй (User Story Map). Данный инструмент позволяет визуализировать функциональность системы в виде двумерной матрицы, где горизонтальная ось представляет этапы взаимодействия пользователя с продуктом, а вертикальная — порядок выпуска функциональных модулей.

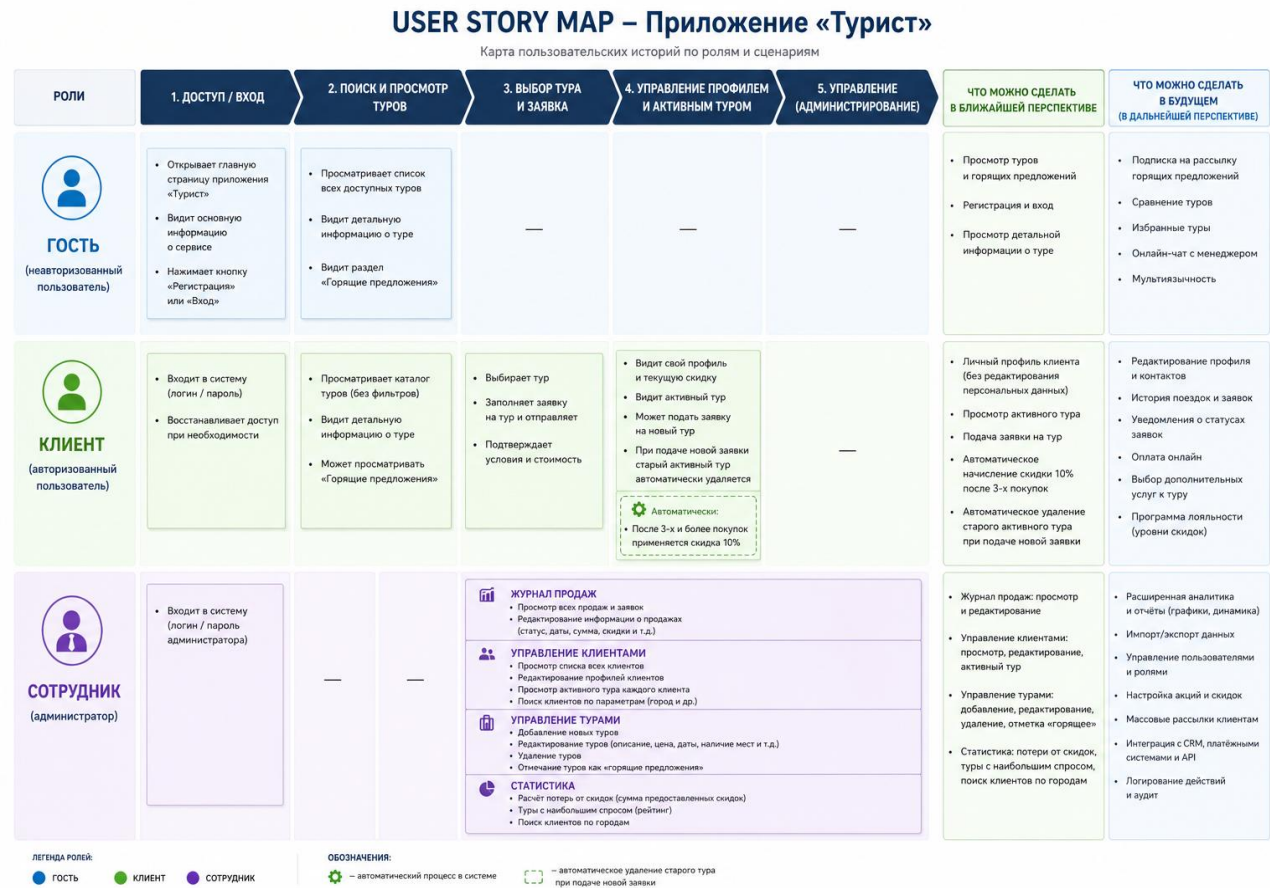


Рисунок 2.2.1 — UserStoryMap диаграмма

Диаграмма User Story Map для приложения «Турист» описывает взаимодействие трёх ролей пользователей с системой: гостя, клиента и

сотрудника. Гость может открыть главную страницу, просматривать список туров и раздел с горящими предложениями, а также перейти к регистрации или входу. Клиент после авторизации получает доступ к каталогу туров и может выбрать подходящий тур для оформления заявки. В системе предусмотрено автоматическое удаление старого активного тура при подаче новой заявки. Также клиент видит информацию о своей скидке и текущем активном туре. После трёх и более покупок система автоматически начисляет скидку 10%. Сотрудник системы выполняет административные функции и управляет продажами, клиентами и турами. Администратор может добавлять, редактировать и удалять туры, а также отмечать их как горящие предложения. В разделе статистики сотрудник анализирует потери от скидок, популярность туров и выполняет поиск клиентов по городам. Дополнительно диаграмма показывает перспективы развития системы, включая онлайн-оплату, программу лояльности, интеграцию с CRM и расширенную аналитику.

2.2.2 Диаграмма UseCase

На рисунке 2.2.2 представлена диаграмма вариантов использования (UseCase), которая формализует функциональные требования к системе и определяет границы ответственности для каждой категории пользователей.

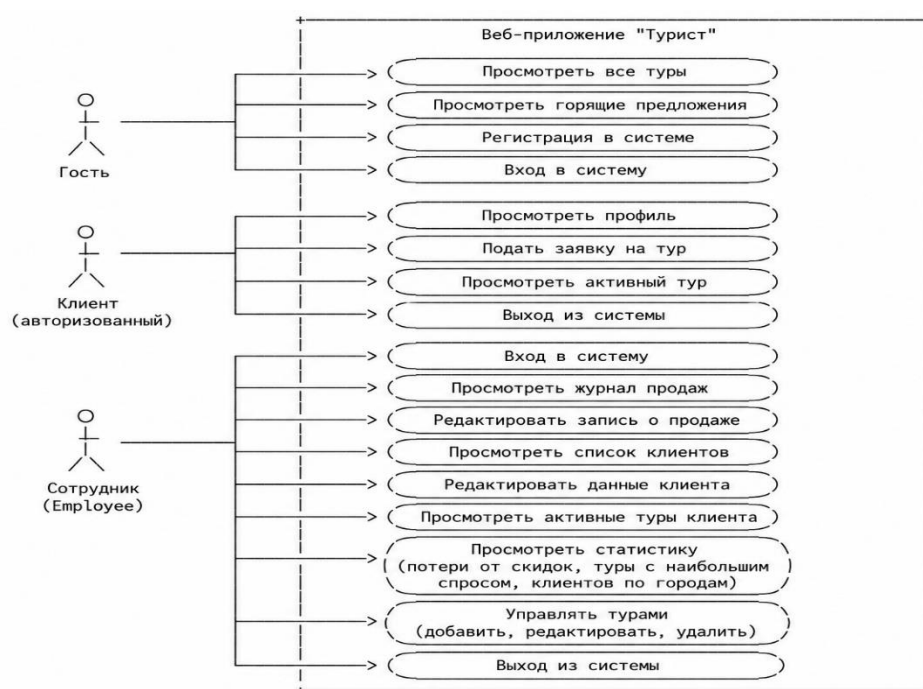


Рисунок 2.2.2 — UseCase диаграмма

Данная диаграмма описывает ролевую модель и сценарии использования системы туристического агентства, где все действия распределены между гостем, авторизованным клиентом и работником.

Гость обладает минимальными возможностями: он может только просматривать список «горящих» путевок (с близкой датой вылета) и оставить заявку на любую путевку без необходимости входа в систему, причём действие «Оставить заявку на путевку» указано дважды, что, вероятно, подчёркивает его ключевую роль для неавторизованных пользователей или дублирует разные способы оформления заявки.

Клиент (после авторизации) получает дополнительные права: он может изменить собственные данные, заблокировать себя (что звучит нестандартно – возможно, речь о временной заморозке профиля или отказе от услуг), а также просматривать путевки на заданную дату, статистику по выбору путевок, информацию о потерях из-за скидок и самую дорогую путевку.

Работник (менеджер или администратор) наделён наиболее широкими полномочиями: он добавляет, удаляет и изменяет путевки, просматривает и редактирует журнал продаж, видит все те же отчётные данные (потери от скидок, самая дорогая путевка, статистика выбора, путевки на дату), а также управляет клиентами (изменяет их данные) и может заблокировать клиента, что отличается от самоблокировки клиентом.

Таким образом, диаграмма чётко разделяет уровни доступа: гость — только просмотр и заявки, клиент — персональные действия и часть аналитики, работник — полный цикл управления путевками, продажами и клиентскими записями.

2.2.3 ER-диаграмма

ER-диаграмма представляет структуру базы данных информационной системы туристического агентства. Основной сущностью системы является

таблица `clients`, в которой хранится информация о клиентах: ФИО, номер телефона, паспортные данные, скидка и количество приобретённых туров.

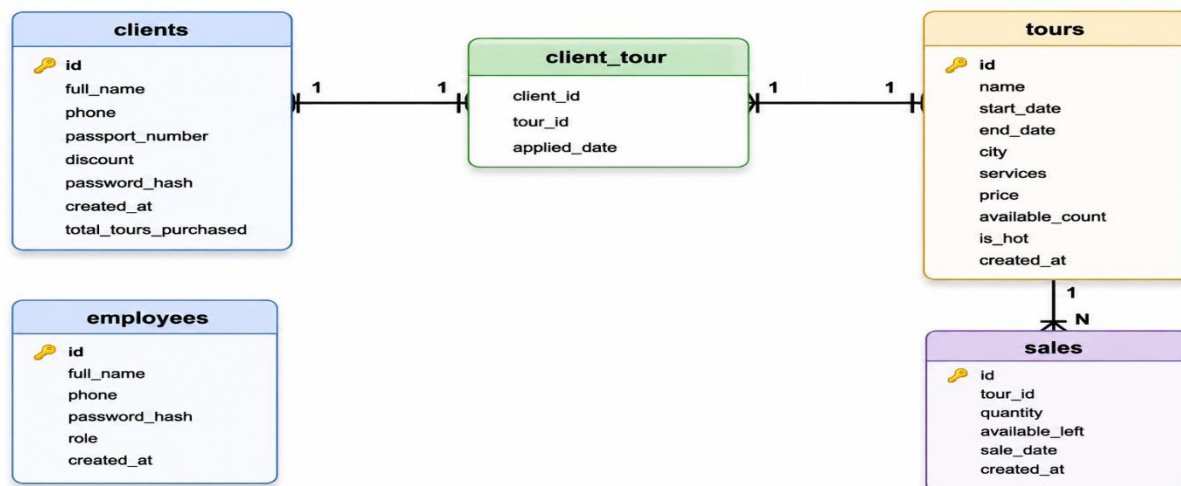


Рисунок 2.2.3 — ER-диаграмма

Таблица `tours` содержит сведения о туристических путёвках, включая название тура, даты начала и окончания, город, стоимость и количество доступных мест. Для хранения информации о продажах используется таблица `sales`, в которой фиксируются данные о количестве проданных туров и оставшихся местах. Таблица `employees` предназначена для хранения данных о сотрудниках агентства, таких как ФИО, телефон, роль и пароль для авторизации.

Связь между таблицами `clients` и `client_tour` является связью типа 1:1, что означает соответствие одной записи клиента одной записи заявки на тур. Аналогичная связь 1:1 установлена между таблицами `client_tour` и `tours`, где каждой заявке соответствует один определённый тур. Таблица `client_tour` выступает промежуточной сущностью и хранит дату оформления заявки клиента на тур. Между таблицами `tours` и `sales` установлена связь 1:N, так как один тур может участвовать во множестве продаж. Разработанная ER-диаграмма позволяет наглядно представить структуру базы данных и упрощает дальнейшую разработку и сопровождение информационной системы.

2.2.4 Описание архитектуры фронтенда

Клиентская часть реализована в виде одностраничного приложения (SPA) на базе библиотеки React 18 с использованием TypeScript и инструмента сборки

Vite. Архитектура фронтенда построена по принципу разделения ответственности и организована в следующие логические слои.

Слой компонентов. Пользовательский интерфейс разбит на два уровня абстракции. Директория `components/ui/` содержит атомарные примитивы: `Button`, `Input`, `Modal`, `Tabs`, `QtyStepper`, `Price`, `CartPill`, `Select`, `Radio` и другие. Директория `components/shared/` содержит составные (композиционные) компоненты, формирующие устойчивые блоки UI: `Header`, `Footer`, `ProductCard`, `ProductModal`, `CartDrawer`, `AuthModal`, `ReviewSection`. Такое разделение позволяет переиспользовать примитивы независимо и упрощает сборку сложных экранов из готовых блоков.

Слой страниц. Директория `pages/` содержит компоненты верхнего уровня, соответствующие маршрутам приложения: `HomePage`, `CatalogPage`, `CheckoutPage`, `OrderSuccessPage`, `AccountPage`, `AdminPage`. Маршрутизация организована через `React Router v6` с использованием компонента `BrowserRouter`; корневой `AppShell` выстраивает единую оболочку приложения (шапка, подвал, оверлей корзины) и монтирует страницы через `<Routes>`.

Слой управления состоянием. В приложении применяется двухуровневое управление состоянием. Глобальное клиентское состояние хранится в `Zustand`-сторах: `cart.ts` управляет локальной корзиной с персистентностью через `localStorage` (`middleware persist`, ключ `aw-cart`); `auth.ts` хранит данные текущего пользователя и `access`-токен в памяти. Серверное состояние (данные каталога, заказы, профиль) кэшируется и синхронизируется через `TanStack Query`, что исключает избыточные сетевые запросы и обеспечивает инвалидацию кэша при мутациях.

Слой сетевого взаимодействия. Директория `api/` содержит доменно-организованные модули запросов: `auth.ts`, `products.ts`, `cart.ts`, `orders.ts`, `users.ts`, `admin.ts`, `reviews.ts`, `promo.ts`. Все запросы проходят через централизованный `axios`-инстанс (`api/client.ts`), который реализует два `interceptor`-а: первый автоматически подставляет `Bearer`-токен из `Zustand`-стора в заголовок `Authorization`, второй перехватывает ответы с кодом `401`, инициирует тихое

обновление access-токена через `/api/auth/refresh` (httpOnly cookie), ставит в очередь параллельные запросы до завершения refresh-а и повторяет их с новым токеном. При неудаче refresh-а пользователь автоматически разлогинивается.

Дизайн-система. Все визуальные константы централизованы в файле `tokens.css` в виде CSS custom properties: цветовая палитра, типографическая шкала, отступы на базе 4px-сетки, радиусы скругления. Шрифт — Manrope (Google Fonts). Это обеспечивает единообразие интерфейса и упрощает глобальные стилистические изменения.

2.2.5 Описание архитектуры бекенда

Серверная часть реализована на платформе Node.js с использованием фреймворка Express и языка TypeScript. Взаимодействие с базой данных осуществляется через ORM Prisma с PostgreSQL в качестве хранилища. Исходный код сервера организован в директории `src/` и разделён на следующие логические уровни.

Слой маршрутизации (`routes/`). Каждый функциональный домен выделен в отдельный Express-роутер: `auth`, `products`, `artists`, `categories`, `cart`, `orders`, `users`, `admin`, `promo`. Роутеры подключаются в точке входа `index.ts` с соответствующими префиксами `/api/*`. Такой подход обеспечивает изоляцию доменов и упрощает навигацию по кодовой базе.

Слой сервисов (`services/`). Комплексная бизнес-логика, требующая переиспользования, вынесена в сервисы.

Модуль `auth.ts` инкапсулирует регистрацию, аутентификацию, ротацию refresh-токенов и выход из системы. В маршрутах с более простой логикой взаимодействие с базой данных осуществляется непосредственно через Prisma-клиент.

Слой middleware. Входящие запросы проходят через цепочку промежуточных обработчиков. `helmet` добавляет защитные HTTP-заголовки. `cors` ограничивает источники запросов адресом клиентского приложения. `authenticate` проверяет Bearer JWT-токен в заголовке `Authorization` и прикрепляет

расшифрованный payload к объекту запроса. optionalAuth применяется в публичных эндпоинтах, где авторизация желательна, но не обязательна. requireRole реализует ролевую модель доступа (RBAC): все маршруты /api/admin/* доступны исключительно пользователям с ролью ADMIN. errorHandler обеспечивает единообразную обработку ошибок: доменные исключения (AppError) сериализуются в JSON с соответствующим HTTP-статусом, непредвиденные ошибки возвращают статус 500.

2.2.6 Список API

Проектирование Взаимодействие между клиентской и серверной частями системы построено на архитектурном стиле REST с использованием формата JSON для обмена данными. API разделён на функциональные домены, обеспечивающие полный жизненный цикл работы с магазином: от аутентификации до администрирования.

Защищённые эндпоинты требуют наличия валидного JWT-токена в заголовке Authorization: Bearer <token>, административные эндпоинты дополнительно проверяют роль ADMIN.

В таблице ниже приведены основные методы API, реализованные в системе

Таблица 2.5.2 — Список API

Метод	Эндпоинт	Описание операции	Тело / Параметры
GET	/api/admin/orders	Все заказы с фильтрацией и пагинацией	Query: status, page + JWT ADMIN
GET	/api/admin/products	Полный список товаров для администратора	JWT + роль ADMIN
GET	/api/admin/promo	Список всех промокодов	JWT ADMIN
GET	/api/admin/stats	Статистика продаж: выручка, заказы по месяцам, по статусам	JWT ADMIN
GET	/api/admin/users	Список пользователей с пагинацией	JWT ADMIN
GET	/api/artists	Список художников-авторов	—
GET	/api/cart	Серверная корзина авторизованного пользователя	JWT

GET	/api/categories	Список категорий товаров	—
GET	/api/orders	История заказов текущего пользователя	JWT
GET	/api/users/favorites	Список избранных товаров	JWT
GET	/api/users/me	Профиль текущего пользователя	JWT
GET	/api/users/orders	История заказов в личном кабинете	JWT
POST	/api/admin/products	Создание нового товара	{ "slug", "title", "price", "categoryId", "artistId" } + JWT ADMIN
POST	/api/admin/promo	Создание промокода	{ "code", "discountPercent", "expiresAt?" } + JWT ADMIN
POST	/api/auth/login	Аутентификация по email или телефону	{ "email", "password" }
POST	/api/auth/logout	Завершение сессии, аннуляция токена	JWT в заголовке
POST	/api/auth/refresh	Обновление access-токена	httpOnly cookie refresh_token
POST	/api/auth/register	Регистрация нового пользователя	{ "email", "password", "name?" }
POST	/api/cart	Добавление позиции в корзину	{ "productId", "quantity?", "color?" } + JWT
POST	/api/orders	Оформление заказа (гость или авторизованный)	{ "contactName", "contactEmail", "contactPhone", "deliveryType", "deliveryAddress?", "items", "promoCode?" }
POST	/api/products/:slug/reviews	Создание отзыва (только для покупателей)	{ "rating", "text" } + JWT
POST	/api/promo/validate	Проверка промокода перед применением	{ "code" }
POST	/api/users/favorites/:productId	Добавление товара в избранное	JWT
PATCH	/api/admin/orders/:id/status	Смена статуса заказа и добавление комментария	{ "status", "statusComment?" } + JWT ADMIN
PATCH	/api/admin/products/:id	Частичное редактирование товара	Поля товара (любые) + JWT ADMIN
PATCH	/api/admin/products/:id/images/:imageId/primary	Назначение основного изображения товара	JWT ADMIN
PATCH	/api/admin/promo/:id	Активация или деактивация промокода	{ "isActive" } + JWT ADMIN
PATCH	/api/admin/users/:id/block	Блокировка или разблокировка пользователя	{ "isBlocked" } + JWT ADMIN
PATCH	/api/cart/:itemId	Изменение количества позиции	{ "quantity" } + JWT

PATCH	/api/users/me	Обновление профиля, смена пароля	{ "name?", "phone?", "currentPassword?", "newPassword?" } + JWT
DELETE	/api/admin/products/:id	Удаление товара (при отсутствии истории заказов)	JWT ADMIN
DELETE	/api/admin/products/:id/images/:imageId	Удаление изображения	JWT ADMIN
DELETE	/api/cart/:itemId	Удаление позиции из корзины	JWT
DELETE	/api/users/favorites/:productId	Удаление товара из избранного	JWT
PUT	/api/cart/sync	Слияние локальной корзины с серверной при входе	{ "items": [{ "productId", "quantity", "color" }] } + JWT

2.2.5 Макет интерфейса

Первоначальная страница содержит горячие путевки и все путевки вместе. Клиент заходит на сайт и видит предложенные туры. Также на этой странице он может подать заявку на участие в определенной программе.

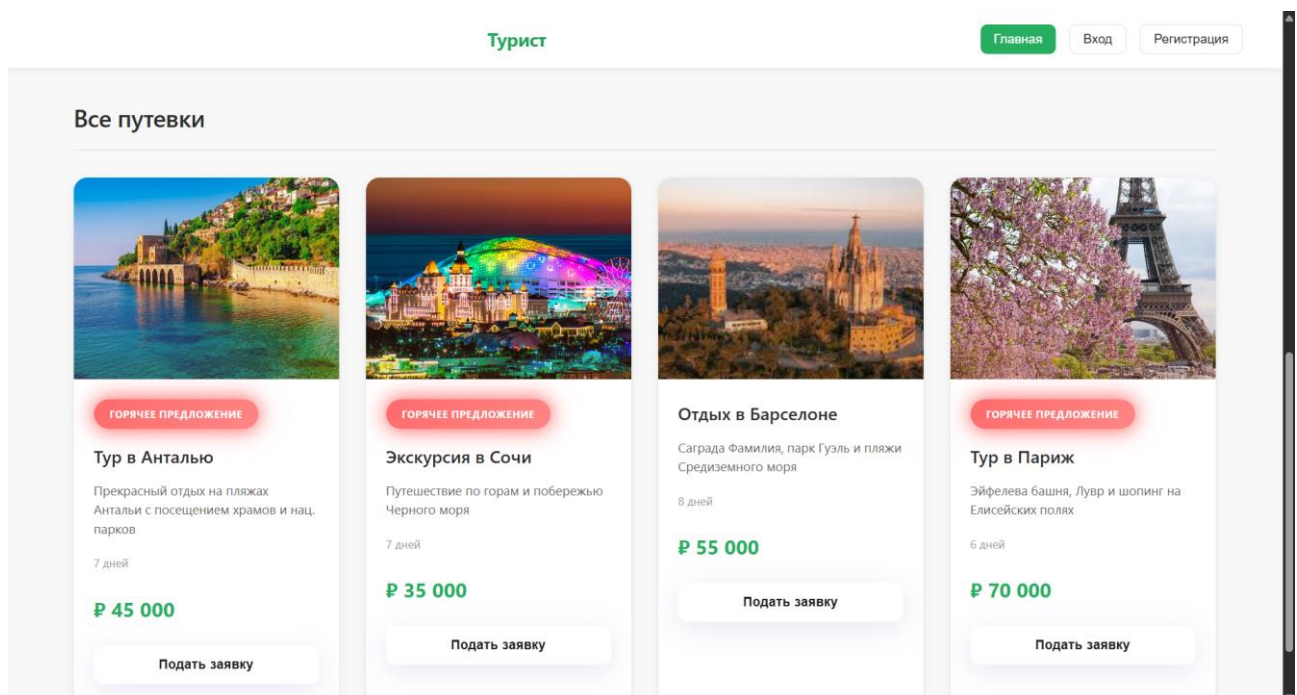


Рисунок 2.2.4 — страница с путевками

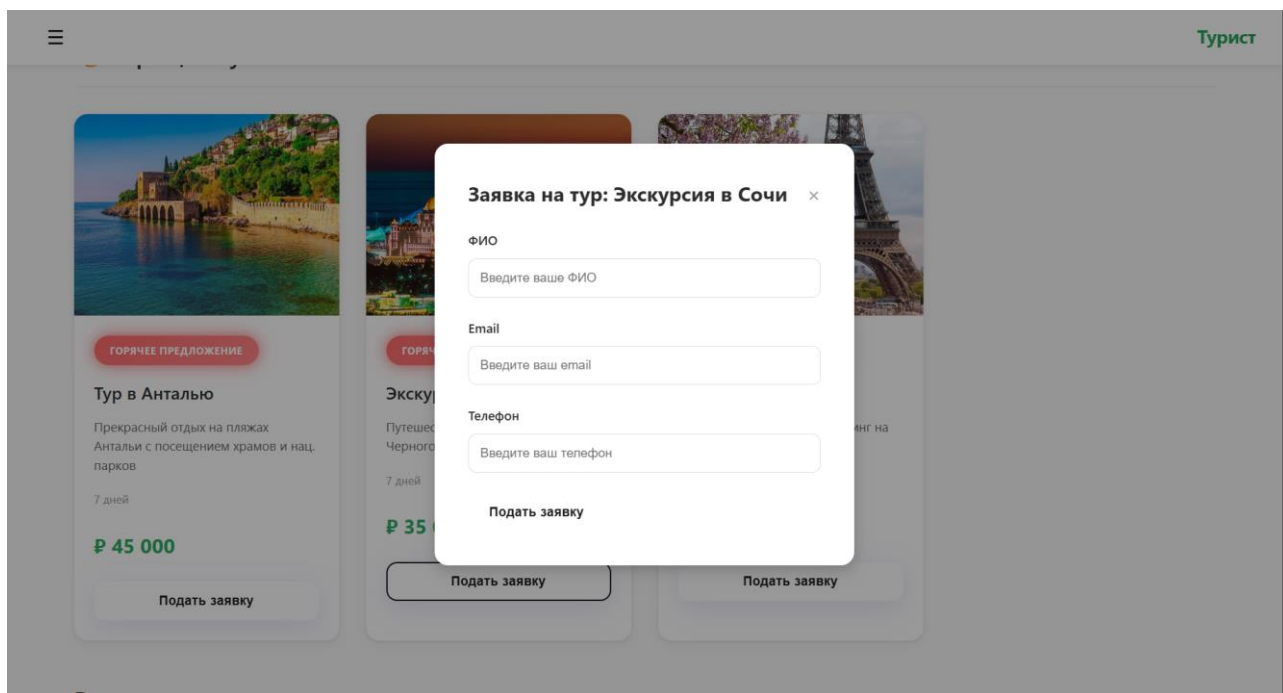


Рисунок 2.2.5 — страница с путевками: форма подачи заявки

На странице авторизации пользователь имеет возможность ввести свои данные для входа в аккаунт, а именно: номер телефона и пароль от уже созданного аккаунта. Если аккаунта нет то пользователь должен нажать на ссылку Зарегистрироваться и создать его.

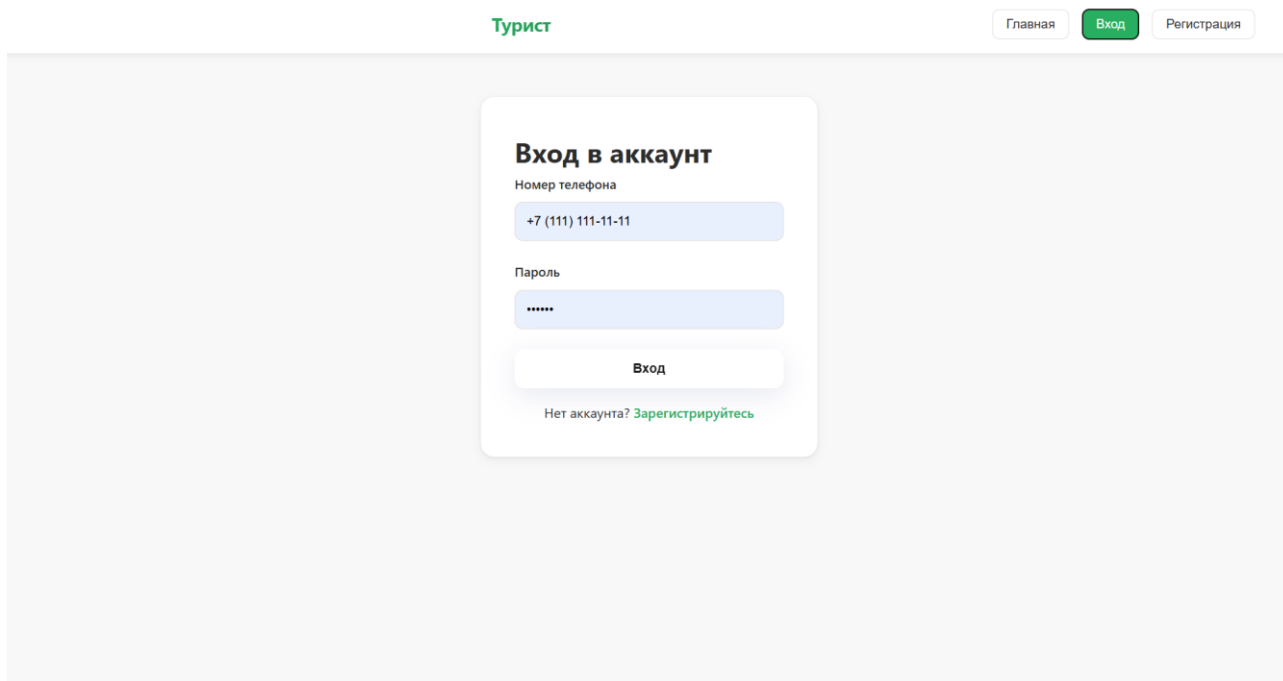


Рисунок 2.2.6 — страница входа

Для работников турфирмы создан отдельный интерфейс. У работников есть доступ к журналу продаж, который представлен на отдельной странице в виде таблицы которую можно редактировать.



Журнал продаж

ID Тура	Количество	Осталось	Дата продажи	Действия
2	1	0	19.05.2026	Редактировать Удалить
3	1	0	19.05.2026	Редактировать Удалить
3	1	0	19.05.2026	Редактировать Удалить
1	1	0	19.05.2026	Редактировать Удалить
2	1	0	19.05.2026	Редактировать Удалить

Рисунок 2.2.7 — журнал продаж

Также у работников есть доступ к журналу клиентов. На этой странице представлена таблица с клиентами и их данными для быстрого доступа к информации и возможности ее изменения.



Журнал клиентов

ФИО	Телефон	Паспорт	Дата регистрации	Мой тур	Куплено путевок	Скидка (%)	Действия
Артём Зайцев	+7 (111) 111-11-11	123456	19.05.2026	Горы Кавказа	0	10.00	Сохранить Отмена
Второй тест	+7 (222) 222-22-22	-	19.05.2026	Экскурсия по Европе	0	0.00	Редактировать Удалить
Третий тест	+7 (333) 333-33-33	-	19.05.2026	Отдых в Анталье	0	10.00	Редактировать Удалить

Рисунок 2.2.8 — журнал клиентов

Следующая страница позволяет работникам турфирмы просматривать динамические данные и статистику предприятия. Это упрощает пользование системой и позволяет менеджерам и руководителям эффективнее управлять фирмой.

Потери из-за скидок

Общие потери: **₽ 22 000**
 Количество клиентов со скидкой: **2**

Самая дорогая путевка

Название: **Экскурсия по Европе**
 Город: **Париж**
 Цена: **₽ 120000.00**

Путевки с наибольшим спросом (Топ-5)

Название	Город	Цена	Продано
Отдых в Анталье	Анталья	₽ 65000.00	2
Горы Кавказа	Сочи	₽ 45000.00	2

Рисунок 2.2.9 — страница с аналитикой

3 ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 Разработка составных элементов

3.1.1 Пользовательский интерфейс

В практической части реализован пользовательский интерфейс веб-приложения "Турист" для управления туристическими путевками, спроектированный на основе требований, сформулированных в теоретической части. Интерфейс обеспечивает последовательный пользовательский сценарий для двух основных ролей: клиентов (поиск туров, подача заявок) и сотрудников (управление продажами, клиентами, турами и аналитика).

Для неавторизованного пользователя (гостя) предусмотрена главная страница с каталогом всех доступных туров, а также отдельный раздел с "горящими предложениями", выделенными специальным оформлением и гарантирующими наибольшую скидку. Навигационная панель включает кнопки входа и регистрации, обеспечивая быстрый доступ к аутентификации.

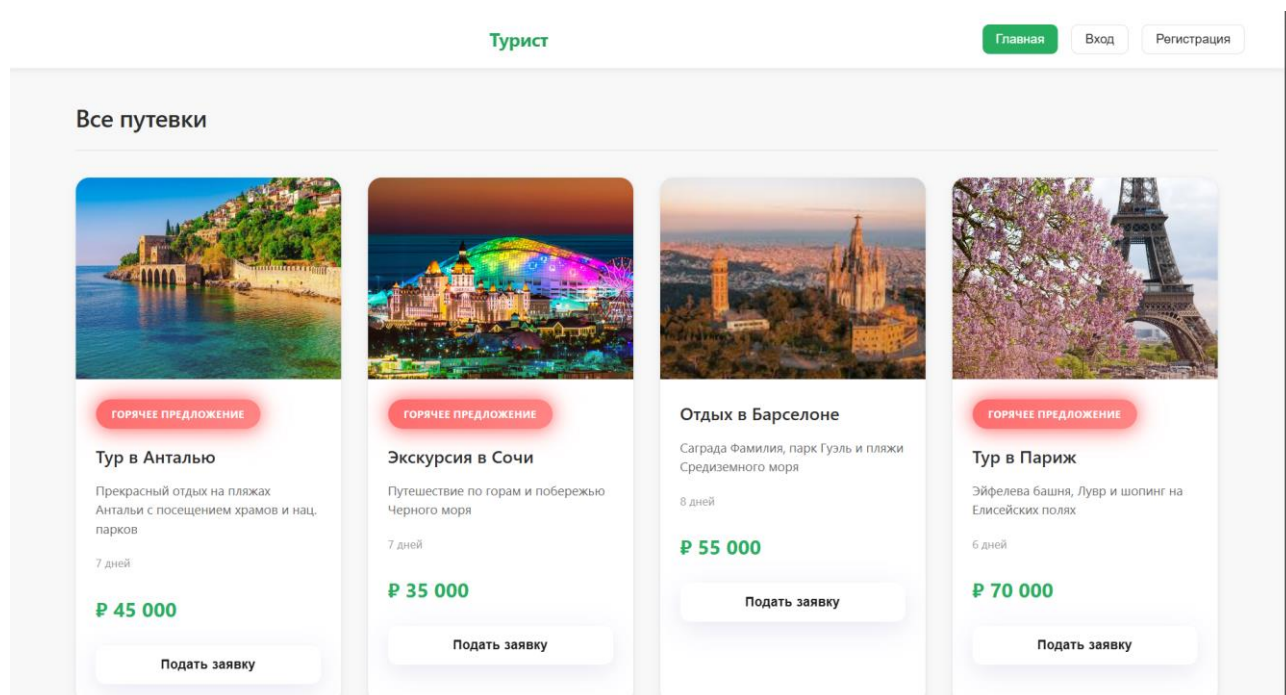


Рисунок 3.1.1 – Главная страница с каталогом туров

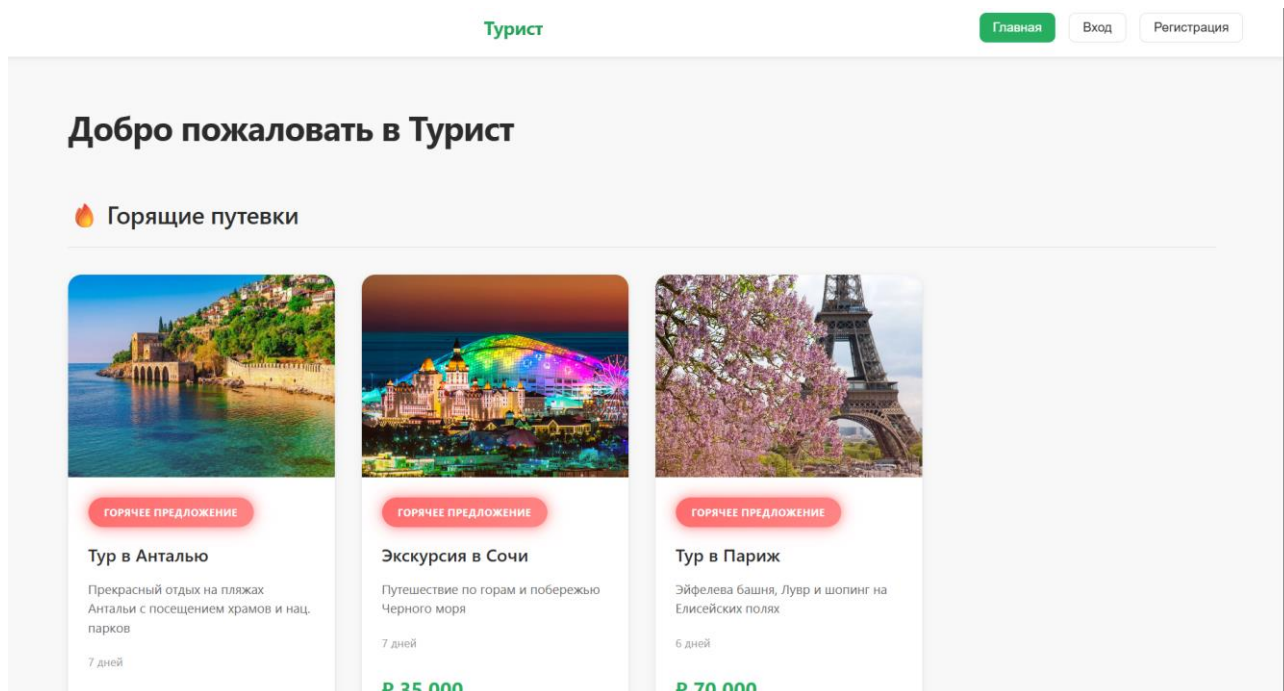


Рисунок 3.1.2 – Раздел "горящих предложений"

После регистрации и входа клиент получает доступ к личному кабинету, где может подавать заявки на туры. При выборе тура открывается модальное окно с подробной информацией о туре и кнопкой подачи заявки.

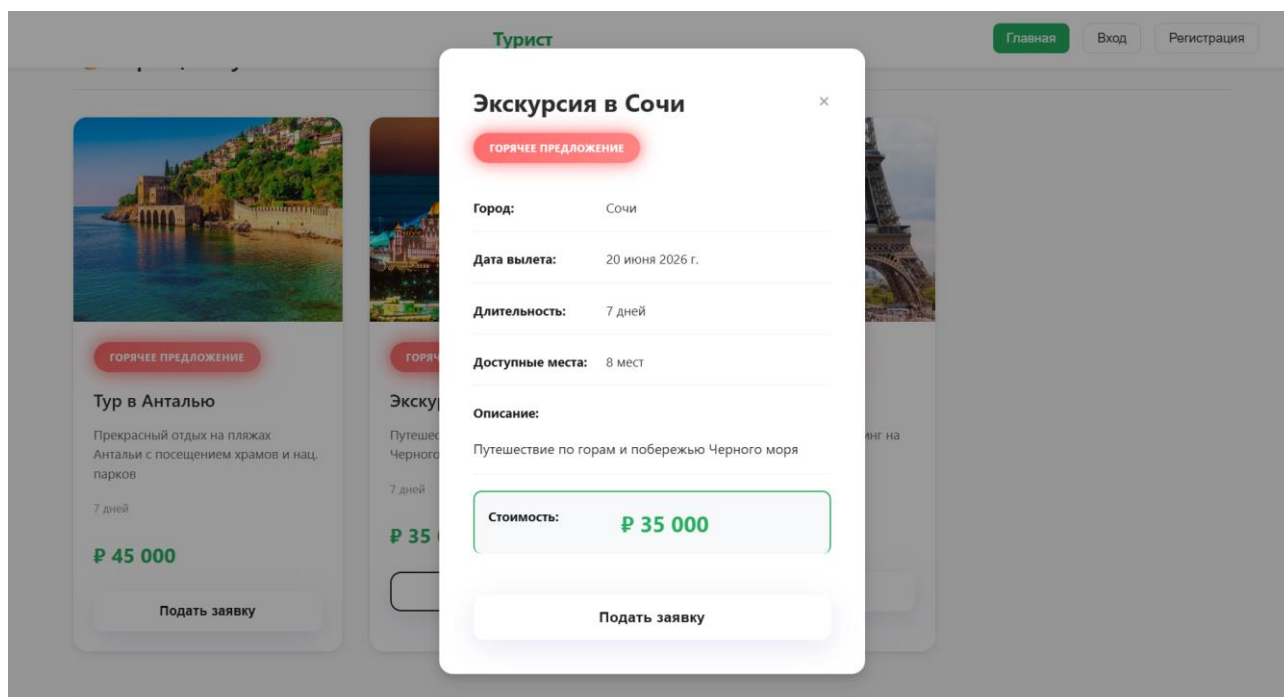


Рисунок 3.1.3 – Форма подачи заявки на тур

Для сотрудника компании предусмотрена административная панель с несколькими разделами. Раздел "Журнал продаж" отображает все оформленные

заказы с возможностью редактирования количества и остатков туров. Раздел "Клиенты" показывает список всех клиентов с их активным туром, информацией о количестве покупок и применяемой скидке, с возможностью редактирования профиля.

☰

Турист

Журнал продаж

ID Тура	Количество	Осталось	Дата продажи	Действия
2	1	0	19.05.2026	Редактировать Удалить
3	1	0	19.05.2026	Редактировать Удалить
3	1	0	19.05.2026	Редактировать Удалить
1	1	0	19.05.2026	Редактировать Удалить
2	1	0	19.05.2026	Редактировать Удалить

Рисунок 3.1.4 – Журнал продаж

☰

Турист

Журнал клиентов

ФИО	Телефон	Паспорт	Дата регистрации	Мой тур	Куплено путевок	Скидка (%)	Действия
Артем Зайцев	+7 (111) 111-11-11	123456	19.05.2026	Горы Кавказа	0	10.00	Редактировать Удалить
Второй тест	+7 (222) 222-22-22	-	19.05.2026	Экскурсия по Европе	0	0.00	Редактировать Удалить
Третий тест	+7 (333) 333-33-33	-	19.05.2026	Отдых в Анталье	0	10.00	Редактировать Удалить

Рисунок 3.1.5 – Управление списком клиентов



Журнал клиентов

ФИО	Телефон	Паспорт	Дата регистрации	Мой тур	Куплено путевок	Скидка (%)	Действия
Артём Зайцев	+7 (111) 111-11-11	123456	19.05.2026	Горы Кавказа	0	10,00	Сохранить Отмена
Второй тест	+7 (222) 222-22-22	-	19.05.2026	Экскурсия по Европе	0	0.00	Редактировать Удалить
Третий тест	+7 (333) 333-33-33	-	19.05.2026	Отдых в Анталье	0	10.00	Редактировать Удалить

Рисунок 3.1.6 – Редактирование данных клиента

Раздел "Управление турами" позволяет добавлять новые туры, редактировать существующие (название, город, дату, цену), управлять доступным количеством мест. Раздел "Статистика" предоставляет аналитику: расчёт потерь от скидок, определение туров с наибольшим спросом, поиск клиентов по городам и выявление наиболее дорогостоящих туров.



Управление турами

Добавить новый тур

Название	Город	Цена (₽)	Длительность	Дата начала	Дата окончания	Услуги	Доступно мест	Действия
Тур в Барселону	Барселона	85 000	7 дн.	2026-05-13	2026-05-20	Отель 4*, Завтрак, Экскурсии	12	Редактировать Удалить
Отдых в Анталье	Анталья	65 000	7 дн.	2026-05-19	2026-05-26	Отель 5*, All Inclusive	15	Редактировать Удалить
Горы Кавказа	Сочи	45 000	11 дн.	2026-05-24	2026-06-04	Горные походы, Базовый лагерь	20	Редактировать Удалить
Экскурсия по Европе	Париж	120 000	14 дн.	2026-05-31	2026-06-14	Франция, Люксембург, Бельгия	8	Редактировать Удалить

Рисунок 3.1.7 – Управление турами в системе

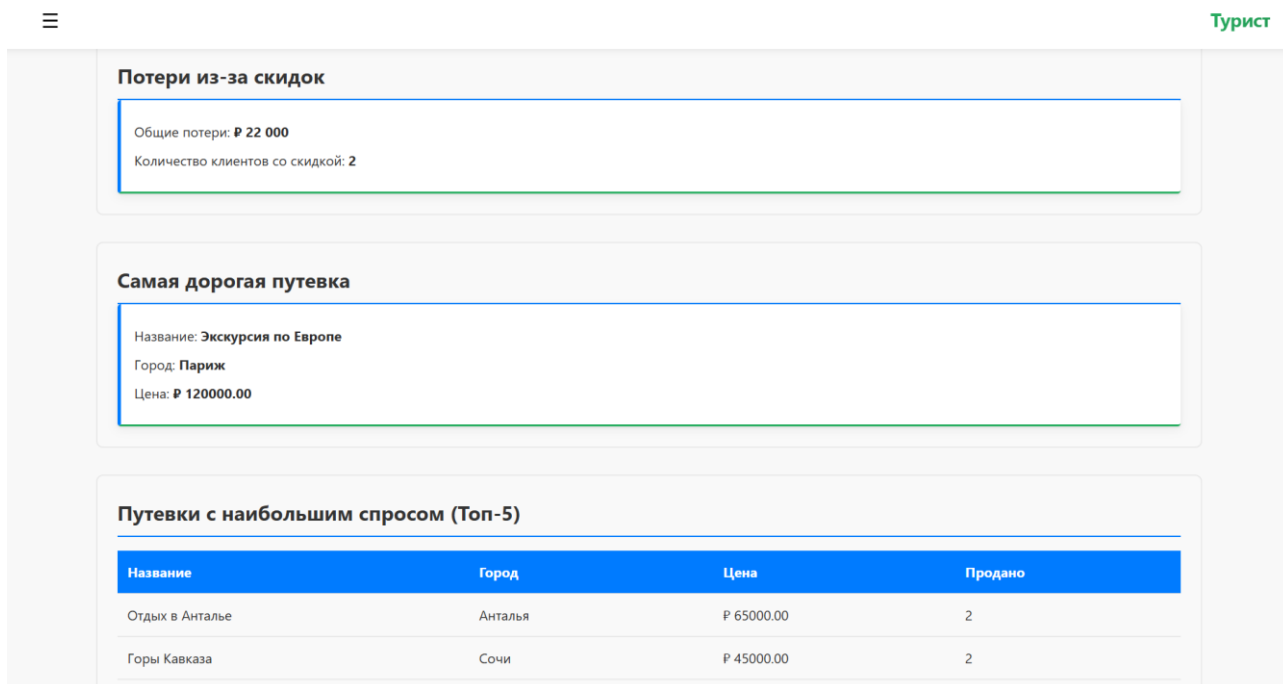


Рисунок 3.1.8 – Статистика: функции

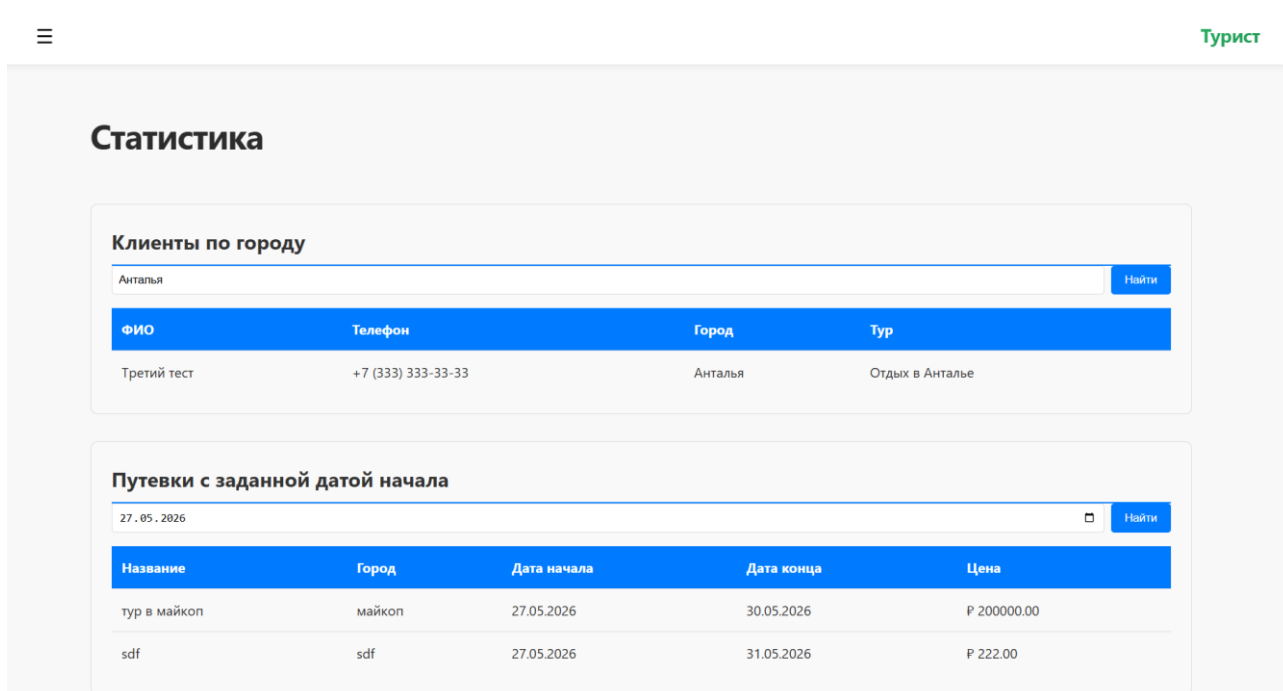


Рисунок 3.1.9 – Статистика: клиенты по городу и путевки по дате

Пользовательский интерфейс разработан с использованием современного дизайн-подхода glass-morphism с премиум цветовой схемой: основной цвет тёмно-серый (#2c2c2c), акцентный зелёный (#27ae60), все элементы имеют закруглённые углы (border-radius 12-16px), кнопки реализованы с эффектом полупрозрачности и размытия (backdrop-filter). Навигационная панель переработана для улучшения видимости и доступности функций, цены туров

выделены зелёным цветом для акцента. Система обеспечивает интуитивный пользовательский сценарий, чёткую иерархию информации и ролевое разграничение доступа.

3.1.2 Реализация фронтенд части

Фронтенд-часть приложения реализована в виде многостраничного веб-приложения на базе React с использованием JavaScript и набора современных инструментов разработки (React Router для маршрутизации, axios для HTTP-клиента, localStorage для локального хранилища). Архитектура клиентского приложения построена на принципе разделения ответственности, обеспечивая модульность и масштабируемость кода.

Корневой компонент App осуществляет маршрутизацию между главной страницей, страницей логина/регистрации, личным кабинетом клиента, административной панелью сотрудника, а также отдельными страницами для каталога, профиля и статистики. Компонентный слой разделён на несколько уровней: примитивы интерфейса (кнопки, поля ввода, модальные окна), составные компоненты (карточки туров, формы заявок, таблицы данных), и полноэкранные страницы.

Управление состоянием реализовано комбинированным способом. Состояние аутентификации (токены доступа, информация о текущем пользователе, его роль) хранится в памяти приложения и синхронизируется с localStorage для восстановления сессии при перезагрузке страницы. Глобальные данные (список туров, информация о клиентах) загружаются с сервера по мере необходимости и кэшируются на клиенте для оптимизации производительности.

Обмен данными с серверной частью реализован через централизованный axios-клиент с автоматической подстановкой JWT access-токена в заголовок Authorization всех запросов. При получении ответа 401 Unauthorized запускается механизм автоматического обновления токена через вызов /api/auth/refresh-token; при успешном обновлении повторно отправляются отложенные запросы.

При неудаче обновления осуществляется корректный выход пользователя из системы.

Стилизация выполнена единым CSS-файлом (App.css) с использованием CSS-переменных (custom properties) для определения цветов, размеров и других параметров оформления. Это обеспечивает единообразие дизайна и упрощает внесение изменений во внешний вид приложения.

Таким образом, фронтенд-часть реализует полный пользовательский сценарий системы управления туристическими путевками, поддерживает ролевую модель (клиент/сотрудник), обеспечивает устойчивую интеграцию с серверной частью и предоставляет удобный, интуитивный интерфейс для обеих категорий пользователей.

3.1.3 Реализация бекенд части

Серверная часть реализована на платформе Node.js с использованием фреймворка Express и языка JavaScript. Архитектура серверного приложения ориентирована на модульность и включает слои маршрутизации, бизнес-логики, middleware и доступа к данным.

В точке входа сервера (server.js) настроены базовые механизмы безопасности и взаимодействия:

- CORS для ограничения запросов от авторизованного клиентского домена;
- парсинг JSON-запросов;
- обработка загруженных файлов и служебные middleware;
- middleware для проверки JWT-токена и контроля доступа;

Серверная логика распределена по функциональным маршрутам:

- /auth: регистрация и вход пользователей;
- /api/tours: получение списка туров, фильтрация, поиск;
- /api/clients: управление клиентами, редактирование профилей;
- /api/sales: создание и управление продажами;

- /api/statistics: аналитика и отчёты;

Подсистема аутентификации реализует JWT-модель: access-токен короткого срока жизни используется в заголовке Authorization, refresh-токен длительного срока хранится в памяти приложения. При обновлении сессии выполняется переиздание access-токена. Для контроля доступа применяются middleware, проверяющие наличие валидного токена и роль пользователя (client/employee).

Бизнес-логика ключевых процессов реализована в маршрутах:

каталог туров: GET /api/tours возвращает все туры, GET /api/tours/hot возвращает горящие предложения, GET /api/tours/by-date/:date фильтрует по дате;

подача заявки на тур: POST /api/sales создает запись о продаже, автоматически удаляя старый тур клиента перед новым (принцип 1 тур на клиента);

управление клиентами: GET /api/clients возвращает список с информацией об активных турах, PUT /api/clients/:id обновляет профиль;

аналитика: GET /api/loss-from-discounts вычисляет потери от скидок, GET /api/tours/top-demand возвращает туры с наибольшим спросом.

Система скидок реализована на серверной стороне: при каждом обновлении информации о клиенте проверяется количество совершённых покупок, и если оно достигает 3 и более, автоматически применяется 10% скидка. Это предотвращает клиентское манипулирование и обеспечивает честность системы.

Обработка ошибок централизована: ошибки валидации, отсутствия данных, несоответствия прав доступа возвращаются в единообразном формате JSON с указанием кода состояния HTTP и описанием проблемы. Это упрощает обработку ошибок на клиенте.

Таким образом, бекенд-часть обеспечивает полный цикл бизнес-операций системы управления путевками, защищённое управление пользовательскими

сессиями, валидацию бизнес-правил и устойчивое взаимодействие с клиентским приложением.

3.1.4 Реализация базы данных

База данных приложения реализована на PostgreSQL и содержит следующие основные таблицы:

clients — хранение данных клиентов: ФИО, телефон, email, хэш пароля, применяемая скидка, количество выполненных покупок;

employees — данные сотрудников: ФИО, телефон, хэш пароля, роль в системе;

tours — информация о турах: название, город назначения, дата отправления, базовая цена, количество доступных мест, флаг "горячего предложения", дополнительная информация;

sales — журнал продаж: ссылка на клиента, ссылка на тур, дата покупки, количество, итоговая стоимость с учётом скидок, остаток свободных мест на момент продажи;

client_tour – связующая таблица, хранящая информацию о турах, которые выбрал клиент.

Целостность данных обеспечивается системой ограничений: первичными ключами, уникальными индексами, внешними ключами. Например, таблица sales содержит foreign key на clients и tours, что предотвращает создание продаж для несуществующих клиентов или туров. При удалении тура из системы все связанные с ним продажи либо архивируются, либо сохраняются для истории.

Схема БД нормализована до третьей нормальной формы, что обеспечивает минимальную избыточность данных, снижает вероятность аномалий при обновлении и облегчает дальнейшее расширение функциональности.

3.1.5 Интеграция модулей

Интеграция модулей разработанного программного продукта реализована по клиент-серверной модели с единым REST API. Клиентская часть (React) взаимодействует с серверной частью (Node.js/Express) через HTTP-запросы в формате JSON, а сервер, в свою очередь, выполняет валидацию, бизнес-логику и операции доступа к PostgreSQL.

На уровне фронтенда интеграция организована через централизованный axios-клиент, который автоматически добавляет access-токен в заголовок Authorization всех запросов. При получении ответа 401 запускается процедура обновления токена; при успехе отложенные запросы повторяются, при неудаче пользователь выводится из системы.

На уровне сервера интеграция построена через цепочку middleware и функции-обработчики маршрутов. Входящий запрос проходит через middleware парсинга, проверку CORS, при необходимости аутентификацию и проверку ролей, затем передается в соответствующий обработчик маршрута, который выполняет валидацию данных, обращение к БД, применение бизнес-логики и возврат результата.

Сквозные бизнес-сценарии также интегрированы. Например, при подаче заявки на новый тур модуль sales получает данные о клиенте и туре, проверяет наличие старого активного тура (если есть, удаляет его), создает новую запись в БД, пересчитывает скидку клиента и возвращает подтверждение на клиент. Администратор может отслеживать эти операции через раздел статистики.

Таким образом, интеграция модулей обеспечивает согласованное взаимодействие всех компонентов системы в рамках единого протокола и поддерживает полный цикл пользовательских и административных операций.

3.2 Установка и настройка

Установка и настройка программного продукта выполнялась в локальной среде разработки с разделением на клиентскую (frontend) и серверную (backend)

части. Для корректной работы системы использовались Node.js, менеджер пакетов npm и СУБД PostgreSQL.

На первом этапе была подготовлена среда исполнения. После клонирования репозитория установлены зависимости для обеих частей проекта:

Листинг 3.1 – установка зависимостей

```
cd backend  
npm install  
cd ../frontend  
npm install
```

Далее была проверена доступность PostgreSQL и создана база данных для приложения. Для серверной части были заданы параметры окружения в файле .env:

Листинг 3.2 – файл .env

```
DATABASE_URL="postgresql://postgres:password@localhost:5432/tourist_db  
JWT_SECRET="your-secret-key-here"  
JWT_REFRESH_SECRET="your-refresh-secret-key"  
PORT=5000
```

Инициализация базы данных выполнена через SQL-скрипт, создавший все необходимые таблицы (clients, employees, tours, sales) и их связи. Далее в БД были загружены начальные данные: категории туров, примеры туров, тестовые учётные записи клиентов и сотрудников.

Запуск приложения в режиме разработки:

Backend:

Листинг 3.3 - запуск backend

```
cd backend  
npm start
```

Сервер запустится на <http://localhost:5000>

Frontend:

Листинг 3.4 – запуск frontend

```
cd frontend  
npm start
```

Приложение откроется на <http://localhost:3000>

После запуска обеих частей происходит инициализация: клиент загружает список туров с сервера, проверяет наличие сохранённого токена в localStorage, при необходимости вызывает механизм обновления сессии. Сервер при старте проверяет соединение с БД и готовность всех эндпоинтов.

Результатом этапа установки и настройки стало получение полностью работоспособного стенда разработки, в котором корректно функционируют клиентский интерфейс, серверная бизнес-логика и база данных, обеспечивается их устойчивое совместное взаимодействие.

3.3 Тестирование

Тестирование разработанного программного продукта выполнено с применением ручного подхода и включило проверку всех ключевых пользовательских сценариев и административных функций.

Тестирование функционала клиента:

Просмотр каталога туров:

- главная страница загружается корректно с полным списком туров;
- раздел "горящих предложений" показывает только помеченные туры;

Аутентификация:

- регистрация с валидными данными успешна;
- вход с корректным телефон/пароль выполняется успешно;
- отказ при вводе неверного пароля;
- после перезагрузки страницы сессия восстанавливается из localStorage;

Подача заявки на тур:

- форма заявки открывается и отправляется корректно;
- скидка применяется корректно при 3+ покупках;

Тестирование функционала сотрудника:

Журнал продаж:

- отображение всех записей о продажах;
- редактирование количества в продаже;
- обновление остатков туров при редактировании;

Управление клиентами:

- отображение полного списка клиентов с активными турами;

- информация о количестве покупок и скидке актуальна;
- редактирование данных клиента сохраняется в БД;

Управление турами:

- добавление нового тура с корректным сохранением в БД;
- редактирование названия, города, даты, цены;
- отмечание тура как "горящее предложение" работает;
- изменение доступного количества мест отображается клиентам;

Статистика:

- расчёт потерь от скидок выполняется без ошибок;
- отображение туров с наибольшим спросом корректно;
- поиск клиентов по городам находит правильные результаты;
- выявление дорогостоящих туров работает как ожидается;

Тестирование безопасности:

- JWT токены проверяются на всех защищённых маршрутах;
- неавторизованные пользователи не могут получить доступ к административным функциям;
- пароли хешируются и не хранятся в открытом виде;
- попытка подделки токена отклоняется сервером;
- по результатам тестирования все критические функции работают корректно, система устойчива к базовым ошибкам ввода и обеспечивает надлежащий контроль доступа между ролями;

3.4 Ввод в эксплуатацию

Перед развёртыванием заказчику передаются исходные материалы проекта: исходный код (backend и frontend), описание структуры БД, перечень переменных окружения, инструкции по установке и запуску. Одновременно определяется целевая инфраструктура размещения.

Рекомендуется следующая архитектура развёртывания:

1. Сервер для Node.js backend-приложения (облако или VPS).

2. СУБД PostgreSQL (на отдельном хосте или управляемый сервис).
3. Сервер для раздачи frontend-приложения (облако, CDN или тот же VPS).
4. На стороне заказчика регистрируется доменное имя проекта и настраиваются DNS-записи. На оба публичных узла выпускаются TLS-сертификаты, весь трафик переводится на HTTPS.

Развёртывание backend-части:

1. Установка Node.js LTS на сервер.
2. Установка зависимостей npm.
3. Конфигурирование production-переменных окружения (.env).
4. Создание/обновление схемы БД.
5. Запуск приложения через процесс-менеджер (PM2 или systemd).
6. Настройка reverse-proxy (nginx) для проксирования запросов.
7. Ограничение размеров загруженных файлов и настройка CORS.

Развёртывание frontend-части:

1. Production-сборка: npm run build.
2. Публикация каталога dist на выбранной платформе.
3. Настройка fallback-маршрутизации для SPA (все неизвестные пути должны возвращать index.html).
4. Фиксирование адреса production API в конфигурации.

Все выявленные дефекты исправляются и проверяются повторно до достижения согласованного уровня качества.

По завершении тестирования заказчику передаются эксплуатационные доступы, инструкции по резервному копированию БД и восстановлению после сбоев, регламент мониторинга доступности и логирования ошибок.

3.5 Сопроводительная документация

3.5.1 Назначение и область применения

Система "Турист" предназначена для управления продажами туристических путевок и включает: каталог туров; оформление заявок на туры клиентами; администрирование туров, клиентов, продаж и аналитики.

Система применяется в условиях постоянного доступа к сети Интернет и рассчитана на использование через современный веб-браузер.

3.5.2 Руководство клиента

Клиент открывает главную страницу сайта и переходит в каталог туров. Используя поиск и фильтры, находит интересующий тур. После выбора заполняет форму заявки и отправляет заявку.

3.5.3 Руководство сотрудника

Сотрудник входит в систему под учётной записью с ролью employee и получает доступ к административному интерфейсу. Основные функции:

Журнал продаж: просмотр, фильтрация, редактирование записей о проданных турах.

Управление клиентами: просмотр списка, редактирование данных, отслеживание активного тура и количества покупок каждого клиента.

Управление турами: добавление новых туров, редактирование существующих (название, город, дата, цена, доступное количество), отмечание как "горящее предложение".

Статистика: расчёт потерь от скидок, определение туров с наибольшим спросом, поиск клиентов по городам, выявление дорогостоящих туров.

3.5.4 Регламент сопровождения

Для обеспечения стабильной работы системы рекомендуется:

1. Ежедневное резервное копирование базы данных.
2. Регулярный мониторинг доступности сервиса.
3. Проверка логов ошибок.
4. Резервирование перед обновлением версии.
5. Тестирование ключевых сценариев после каждого обновления.

3.6 Вывод

В третьей главе была представлена практическая реализация разработанного программного продукта и подтверждена его готовность к использованию по назначению. В ходе работы последовательно рассмотрены ключевые этапы создания системы: реализация пользовательского интерфейса, разработка фронтенд- и бекенд-частей, проектирование и внедрение структуры базы данных, а также интеграция всех модулей в единую функциональную архитектуру.

Результаты практической части показывают, что разработанное решение обеспечивает полный цикл работы системы управления туристическими путевками: от просмотра каталога и оформления заявок до администрирования туров, управления продажами и аналитики. Подтверждена корректность ролевой модели доступа, механизмов аутентификации и взаимодействия между клиентским приложением, серверной логикой и СУБД.

Дополнительно были рассмотрены вопросы установки, настройки, тестирования системы, а также описан порядок её ввода в эксплуатацию. Подготовленная сопроводительная документация охватывает пользовательский, административный и технический контуры, что позволяет самостоятельную эксплуатацию и сопровождение проекта.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы был последовательно пройден полный цикл создания программного продукта: от анализа предметной области до практической реализации и подготовки решения к внедрению.

В теоретической части была исследована область управления туристическими путевками, выявлены ограничения существующих подходов, обоснована необходимость разработки специализированного веб-приложения для автоматизации продаж и администрирования. На основе анализа были сформулированы цели, задачи и требования к системе.

В проектной части выполнено технологическое проектирование: обоснован выбор технологического стека (React для фронтенда, Node.js/Express для бекенда, PostgreSQL для БД), построены диаграммы, спецификации и архитектурные решения.

В практической части разработана полнофункциональная система: реализованы интерфейсы для клиентов и администраторов, интегрированы все модули, проведено тестирование и подготовка к развёртыванию. Система поддерживает полный сценарий работы: поиск туров, оформление заявок, отслеживание, управление продажами и аналитику.

Разработанное решение соответствует поставленным требованиям, обладает необходимой функциональностью и готово к внедрению и дальнейшему развитию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. React Documentation / [Электронный ресурс] // React. — URL: <https://react.dev> (дата обращения: 25.05.2026)
2. Node.js Documentation / [Электронный ресурс] // Node.js. — URL: <https://nodejs.org/en/docs/> (дата обращения: 25.05.2026)
3. PostgreSQL: The World's Most Advanced Open Source Database / [Электронный ресурс] // PostgreSQL. — URL: <https://www.postgresql.org/docs/> (дата обращения: 25.05.2026)
4. Express.js - Fast, unopinionated web framework for Node.js / [Электронный ресурс] // Express. — URL: <https://expressjs.com/> (дата обращения: 25.05.2026)
5. JWT.io - Introduction to JSON Web Tokens / [Электронный ресурс] // JWT. — URL: <https://jwt.io/introduction> (дата обращения: 25.05.2026)
6. Sommerville, I. Software Engineering: A Practitioner's Approach / I. Sommerville. – Pearson, 2015. – 971 с.
7. Fowler, M. Refactoring: Improving the Design of Existing Code / M. Fowler. – Addison-Wesley Professional, 2018. – 448 с.

ПРИЛОЖЕНИЕ

Приложение А – ссылка на репозиторий с исходным кодом проекта

Приложение А

Ссылка на репозиторий с исходным кодом проекта -
<https://github.com/artzajcew/coursework>